

AI 辅助软件项目全生命周期开发 · 通关实践



第四关：研发冲刺

PR 门禁五环 · 测试护栏 · 安全红线 · 性能精度实测 · 故障演练 · 变更四步 · AI 资产管理

授课对象：本科四年级综合实习（3-4 人小组）

学完本讲，你应当能够

OBJECTIVES

- ▶ **跑门禁**：AI 生成 → 自查 → 队友 Approve → CI 绿 → 合并，五环证据环环留档
- ▶ **建护栏**：核心逻辑测试先行，覆盖率门禁（总体 70% / 核心 85% / 增量 80%）CI 固化
- ▶ **守红线**：口令不明文、SQL 参数化、输入必校验、密钥不入库，高危清零才可合并
- ▶ **做实测**：性能掐表、精度冻结影像集、降级真杀进程 —— 只信实测数据
- ▶ **管变更**：改 spec → 同步 SRS → 改 plan/tasks → 改代码，四步走完再闭环验证
- ▶ **攒资产**：Prompt 模式库复用，翻车记录 ≥ 1 则做五问根因分析

本关铁律：AI 写的代码与人工代码同一标准评审；只看证据，不接受「我觉得没问题」。

本讲导览

6.1	关卡导论	交付物清单 · 冲刺总方法论 · AI 资产
6.2	冲刺启动	就绪清单 · 分支策略 · CI 五道闸
6.3	PR 门禁五环	开工仪式 · 自查 · 评审五查 · 成本测算
6.4	测试先行	先红后绿 · 用例四源链 · 先行/后补策略
6.5	pytest 实操	四技巧 · 覆盖率门禁 · 契约测试
6.6	安全红线	四条底线 · 等保映射 · 红队模式
6.7	性能测试	四纪律 · PE-01 优化闭环
6.8	精度测试	冻结影像集 · 面积加权口径 · CR-001
6.9	故障演练	三板斧 · kill Worker 剧本 · 故障库
6.10	调试与重构	先复现 · 假设验证 · 技术债三问
6.11	敏捷仪式	站会三规则 · 周报四段 · 燃尽预警
6.12	变更管理	CR-001 标本 · 四步流程 · RTM v4 回填
6.13	仓库解剖	五件对齐 · 结构评审看点 · selfcheck
6.14	复盘	全流程时间线 · 缺陷案例十则

体例：每节先讲「人工怎么做」（基本功），再讲「人机协同怎么做」（AI 提速但人担责）。

6.1

关卡导论与冲刺总方法论

学完本节后，你应当能够：

- 任务 4 是全生命周期的研发主体：两个迭代、PR、原子任务
- 过关看证据：PR 五环、覆盖率、实测数据、变更记录，一样不能少

任务 4 交付物清单：六类产物一个冲刺包

交付物与过关条件

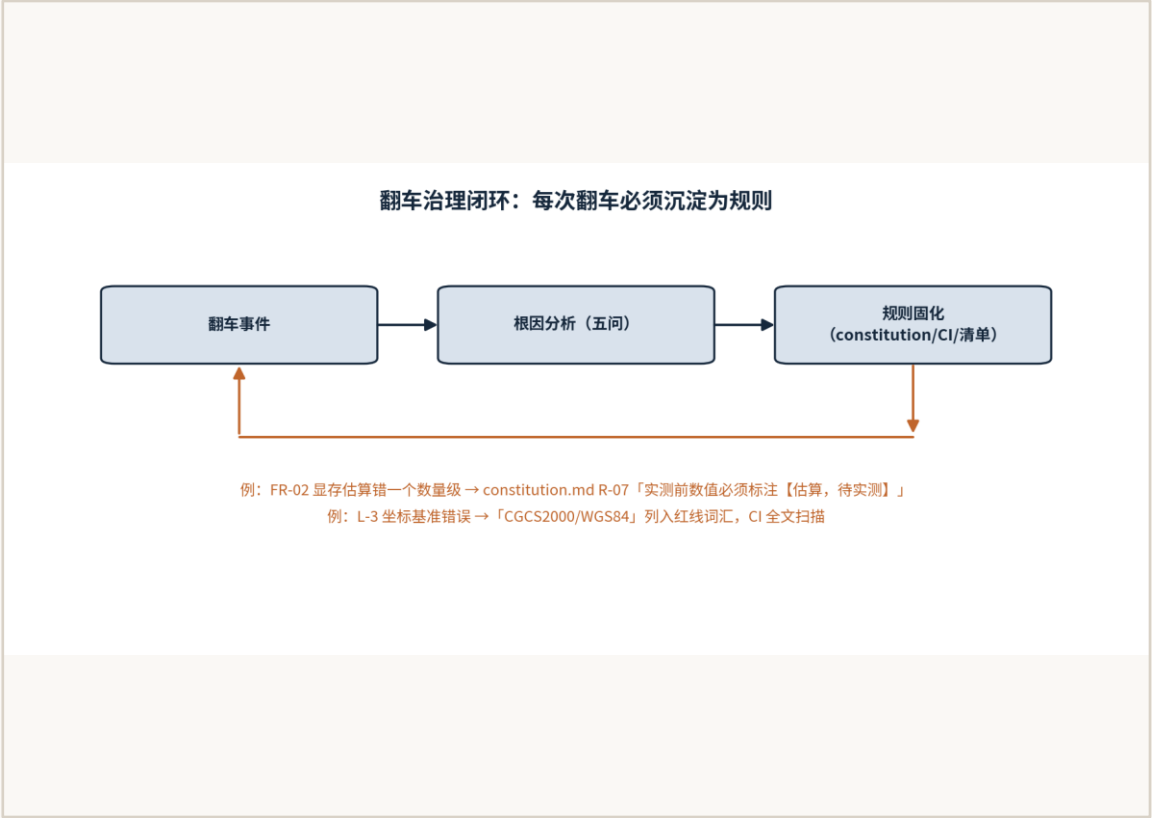
类别	交付物	样例
代码	可运行仓库（含测试、CI、脚本）	sprint_repo/
过程证据	PR 记录（自查+评审+CI证据）	30 号文 PR#47
管理记录	迭代计划与燃尽、站会纪要、周报	27 号文、32 号文
变更记录	变更单（四步流程全链）	31 号文 CR-001
AI 资产	Prompt 模式库、翻车记录	28 号文、29 号文
追踪	RTM v4（代码/PR/测试挂接）	33 号文 xlsx

过关条件：PR 门禁真实运转（抽任一 PR 可讲五环）；覆盖率过门禁、性能精度实测达标、演练至少一轮；变更全部走四步；AI 资产成库且被实际使用。

冲刺怎么组织：四件事开工前定死

翻车治理闭环

- ▶ **implement 段**：spec.md 是 AI 的工作依据，plan/tasks 是人的导航图 —— 代码仓里永远先有任务卡再有代码
- ▶ **开工仪式四件套**：任务卡 + spec 条款 + 接口签名 + 约定/禁止清单，组成 context pack，每次任务现做不用库存
- ▶ **主副修机制**：每人主攻方向（后端/前端/测试/DevOps），必须完成跨方向联调接口
- ▶ **AI 资产沉淀**：好 Prompt 进模式库复用；每次翻车做五问根因分析并固化为规则 —— 翻车记录为零反而扣分



翻车 → 五问根因 → 规则固化（constitution/CI/清单）→ 规则被 AI 遵守

6.2

冲刺启动：环境、分支与 CI 就绪

学完本节后，你应当能够：

- 目标：启动日当天，任何成员提交的 PR 都能被 CI 自动拦住低级错误
- CI 先就绪，代码后开工 —— 顺序反了，护栏就是摆设

启动日就绪清单：六项全绿才许写业务代码

就绪清单 · 分支 · 环境

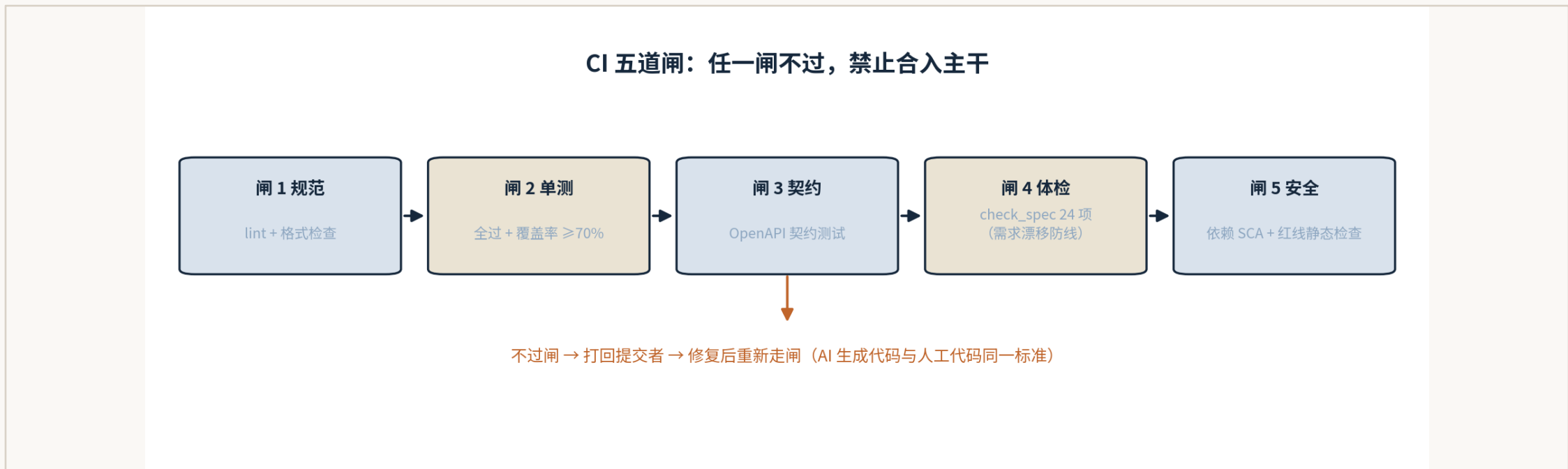
- ▶ **环境一致性**：Python 版本、依赖锁文件、GPU 可选声明写进 README；「我机器上能跑」一律视为环境问题而非代码问题
- ▶ **分支策略**：main 受保护（禁直推），功能分支 task/T-xx-短名 一人一支，PR 合并即删
- ▶ **pre-commit 本地闸**：lint + 红线扫描快速子集，把最低级错误拦在提交前
- ▶ **CI 五道闸配置入库**：.github/workflows 与脚本随仓走，换机器不失效
- ▶ **selfcheck 可运行**：交付自检脚本第一天就建，断言逐迭代增加
- ▶ **看板就位**：tasks.md 原子任务进看板，状态列即燃尽图数据源

检查方式：换一名成员的干净机器，clone → 装环境 → 跑通 selfcheck。

五道闸：快的在前、慢的在后、失败即停

CI 配置即质量防线

- ▶ Gate1lint（ruff）：拦未使用变量、危险语法——秒级
- ▶ Gate2单测+覆盖率：总体 ≥70%、核心 ≥85%——T-22 首推 68% 被真实拦截
- ▶ Gate3/4/5：引用核验拦编造编号；契约拦 11 处 x-roles/x-fr 缺口；安全扫描拦高危 CVE 依赖



澜图样例 CI 流水线：本冲刺共拦截 9 次，每一道闸都有真实拦截记录

CI 失败处置：三条纪律，没有例外

红不合并 · 30 分钟 · flaky 当日

- ▶ **红不合并**：任何红灯 PR 不得合并，「先合进去再修」是技术债的最大来源
- ▶ **30 分钟升级**：自己 30 分钟搞不定的失败，站会上抛出来——卡住不说比失败本身更伤进度
- ▶ **flaky 当日处理**：偶发失败的测试当天定位（时序/并发/外部依赖三选一居多），不许用重跑蒙混——重跑通过的 flaky 测试等于没有测试

反例：某组把偶发失败的精度测试标记 skip「回头再补」，验收前夜集中爆发，返工两天。

6.3

PR 门禁五环与 AI 代码评审

学完本节后，你应当能够：

- 目标：每个合并的 PR 都经得起「抽任一可讲」的抽查
- 单 PR 门禁成本约 25 分钟，换回归缺陷下降一个量级

6.3 五环（上）

环一到环三：生成、自查、队友评审

AI 生成 → 自查 → Approve

- ▶ **环一 开工仪式**：context pack 现做现用（任务卡/spec 条款/接口签名/约定禁止清单），约 40 行 25 分钟 —— pack 随 PR 提交，是提示词质量的检查证据
- ▶ **环二 提交者自查七项**：跑通单测/契约/selfcheck、边界条件、需求编号、AI 参与声明等 —— 本冲刺打回 9/28，首因是自查清单缺项（4 次）
- ▶ **环三 队友 Approve**：≥1 人同标准评审，问题分四类 —— 事实（API 存在吗）、边界（异常处理在吗）、意图（实现的是需求还是 AI 以为的）、数值（常量有出处吗）

评审意见三级：阻断（必须改）→ 建议（说明理由可不改）→ 疑问（答复即可）。

6.3 五环（下）

环四环五与成本收益：门禁不是开销是投资

CI 绿 → 合并 → 归档

- ▶ **环四 CI 绿**：五道闸全绿才解锁合并按钮；红灯处置见 6.2 纪律
- ▶ **环五 合并与归档**：合并后三分钟动作 —— 更新 RTM 行（PR 号入列）、看板挪卡、燃尽刷新
- ▶ **成本测算**：单 PR 门禁约 25 分钟（自查 8 + 评审 12 + CI 等待 5）
- ▶ **收益测算**：拦在 PR 阶段的缺陷修复成本以分钟计；漏到联调/验收阶段以小时甚至天计 —— 样例 CI 拦截 9 次，粗估避免返工 ≥ 3 人日

判断门禁做实的标志：打回率既不为零（走过场）也不畸高（清单没人读），样例 9/28 $\approx 32\%$ 。

评审五查：专抓 AI 代码的系统性偏差

评审清单

查	问题	工具/方法
查事实	引用的 API/编号/参数真实存在？	对照契约与设计文档；check_refs.py
查边界	空值/越界/并发/超时的处理在吗？	异常流清单对照 DLD
查意图	实现的是需求要的，还是 AI 以为的？	逐函数指认 FR 编号（反镀金）
查数值	常量有出处吗（需求条款或实测）？	数值溯源：无出处即打回
查测试	断言真有效吗（恒真断言识别）？	变异思维：改错一处实现，看用例是否变红

AI 预筛分工：评审前让 AI 先过一遍 diff 列疑点，人只审疑点与意图层 —— 评审时间减半，但「意图」与「数值出处」两项必须人看，AI 审不了自己。

AI 代码的五类系统性偏差：评审时戴着这副眼镜

AI 系统性偏差图谱

- ▶ **编造 API**：看似合理实则不存在的调用（D-01 rasterio 事故）—— 对策：官方文档核验入评审规则
- ▶ **常识替代需求**：按「通常做法」而非 spec 约定（阈值 0~100 事故）—— 对策：反常识约定必须进 context pack
- ▶ **调试友好倾向**：日志整体打印、异常吞掉 —— 对策：红线脚本 + 日志必过脱敏层
- ▶ **数值无出处**：超时、重试次数随手填 —— 对策：凡数字必问出处（需求条款或实测）
- ▶ **快乐路径偏好**：异常分支缺失 —— 对策：异常流清单对照 DLD 逐条核

这五类在缺陷案例集中对应 6/10 则 —— 评审清单不是凭空来的，是从翻车记录里长出来的。

PR 门禁全景：同一标准，环环留证

一图总览

AI 生成与人工编写走完全相同的五环；任一环节不过即打回，打回记录本身是过程分证据。



PR 门禁五环（澜图样例：打回 9/28，首因自查清单缺项 4 次）

6.3 课堂练习

练习：当一次评审者

 给出一段 AI 生成的「任务提交 API」代码（60 行，预埋 4 处问题：编造的 queue.push 方法、阈值按 0~100 解析、except 裸捕获）。两人一组，一人评审一人记录。

任务步骤

1. 用评审五查清单逐项过，找出全部 4 处预埋问题并归类
2. 每条意见标注级别：阻断 / 建议 / 疑问
3. 对「数值无出处」一条，写出你要求提交者补充的出处形式
4. 互换角色，用另一段代码再来一轮

完成标准（可检查）

- ✓ 15 分钟内找出 ≥ 3 处并正确归类
- ✓ 意见按 阻断/建议/疑问 三级标注
- ✓ 「数值无出处」写出要求的出处形式
- ✓ 互换角色完成第二轮，记录归档

6.4

测试先行与测试护栏

学完本节后，你应当能够：

- 目标：核心逻辑没有「裸奔」代码，先红后绿成为肌肉记忆
- 测试护栏 CI 固化：不依赖自觉，依赖流水线

6.4 先红后绿

先红后绿的标准节奏与用例四源链

红 → 绿 → 重构

- ▶ **节奏**：从 AC 写失败用例（红）→ 最小实现（绿）→ 重构（仍绿）—— 先写实现后补测试，用例只会证明「代码是错的也能过」
- ▶ **用例四源链**：spec.md 的 Given/When/Then（主源）→ DLD 异常流清单（补充）→ 缺陷史（回归）→ AI 穷举反例（扩边界）
- ▶ **AI 补测工作法**：实现冻结后让 AI 穷举边界用例，人工筛选—— 样例 AI 建议 41 个用例采纳 29 个（71%），否决原因是重复与不可判定
- ▶ **用例质量五条**：断言来自需求而非实现；含恒真断言检测；命名说场景；docstring 挂 AC 编号；异常分支与正常分支成对

恒真检测法：把实现故意改错一处，用例不变红的，等于没用例。

哪些必须先行，哪些可以后补

策略表

类别	策略	理由
核心算法（分块拼接/矢量化/面积统计）	强制先行	逻辑密集，回归成本最高
状态机与调度（GpuQueue）	强制先行	非法迁移必须有用例看守
API 端点	契约测试先行，业务用例同 PR	契约是前后端并行开发的前提
前端交互	关键路径后补，演示前补齐	投入产出比低，用演示脚本兜底
胶水代码（配置/启动）	可后补	CI 冒烟覆盖

「后补」不是「不补」：后补清单进 tasks.md 带负责人与期限，演示前清点。护栏 CI 固化三件套：coverage_gate.py + contract_check.py + 夜间构建报告映射 RTM。

6.5

pytest、契约测试与覆盖率门禁实操

学完本节后，你应当能够：

- 目标：会写四类专业用例，会读覆盖率报告，会让接口始终对上 OpenAPI
- 教具：sprint_repo/tests 目录 —— 每个技巧都有可运行的样例

pytest 四个进阶技巧：样例仓里都有

fixture · 参数化 · 状态机 · 时间

- ▶ **fixture 与依赖注入**：用 fakeredis 替身隔离外部服务；fixture 做环境准备/清理，用例只写三行（准备输入/调用/断言）
- ▶ **参数化反例组**：@pytest.mark.parametrize 把一类边界写成表——非法输入 12 种一行一条，新增反例只加一行
- ▶ **状态机穷举**：Task 状态机的非法迁移逐个断言抛错（queued→done 直接跳？必须拒绝）——D-05 死信事故的护栏
- ▶ **时间敏感逻辑**：心跳超时、退避（30/60/120s）用 freezegun 冻结时钟测，绝不用 sleep 等真实时间

无 pytest 环境的降级方案：unittest + 覆盖率工具同理可用，纪律不变。

覆盖率门禁：70% 是地板，不是天花板

总体 70 / 核心 85 / 增量 80

- ▶ 三层门禁：总体 $\geq 70\%$ （地板）、核心模块 $\geq 85\%$ （逻辑密集区）、增量 $\geq 80\%$ （每个 PR 新增代码单独考核）
- ▶ 门禁三步定法：先测现状定地板 $\rightarrow$ 核心模块名单人工圈定 $\rightarrow$ 增量门禁防「新代码裸奔」—— PR#47 增量 76.2% 被拦，补 3 个异常分支后 89.7% 转绿
- ▶ 报告怎么读：看未覆盖行号分布 —— 集中在 except 块说明异常路径没测；集中在主干说明用例根本没跑起来
- ▶ 心智纠正：覆盖率是「测过哪些地方」的地图，不是「测得好不好」的成绩单 —— 100% 覆盖照样可以零有效断言

缺口不大时补测试约 1 小时 —— 增量门禁设 80% 是「够得着但不放水」的实证。

覆盖率注水四手法：评审与 CI 联合识别

注水识别与处置

注水手法	特征	识别方法	处置
空断言测试	只调用不断言，或 <code>assert True</code>	抽查高覆盖文件的断言密度	打回重写，计一次评审拦截
镜像测试	把实现抄进断言当期望值	断言字面量与实现逐字相同	打回，期望值必须来自 <code>spec</code>
只测快乐路径	全是有效输入，无异常分支	<code>except</code> 块覆盖行号全空	要求补异常用例
<code>import</code> 式覆盖	<code>import</code> 了模块就算覆盖	用例数/覆盖率比值异常	异常即查

变异思维是总钥匙：把实现改错一处，看测试是否变红。不变红的用例，覆盖率数字再漂亮也是注水的。

契约测试：让接口始终对上 OpenAPI

contract_check.py 实操

- ▶ **双向比对**：contract_check.py 枚举代码路由与文档 paths 求差集，逐路径比对方法、必填参数、响应 schema、x-roles/x-fr 标注
- ▶ **五类偏差**：路径不一致 / 方法不一致 / 必填参数缺失 / 标注缺失 / 响应结构漂移 —— 人工核对难度高、量大易漏，正是脚本的价值区
- ▶ **真实事故**：样例曾扫出 11 处 x-roles/x-fr 标注缺口 —— 纯靠自觉必漏，脚本强制扫描才清零
- ▶ **文档先行**：契约与代码冲突时先改 OpenAPI 评审通过再改代码；脚本误报（路径写法差异）做归一化并写进编码规范
- ▶ **人机分工**：AI 建议契约断言与差异修复，人工确认后入库；脚本修改与业务代码同标准评审

D-02/D-03 两则联调事故的共性：单端测试都过，只有契约层能抓 —— 契约测试是前后端并行开发的前提。

6.6

安全红线：不可协商的底线

学完本节后，你应当能够：

- 目标：红线逐条会查会防，`security_redline.py` 进 CI
- AI 时代安全红线更重要 —— 公开语料里的教学代码写法直接进了你的仓库

6.6 红线

红线逐条讲：每条都有真实翻车

根据安全要求，常见：口令 · SQL · 输入 · 密钥

- ▶ R1 口令不明文：存储哈希、日志脱敏——D-09 登录失败日志整体打印请求体，password 明文落盘（AI 的「调试友好」写法）
- ▶ R2 SQL 参数化：禁止字符串拼接——AI 生成查询代码时拼接是高频惯性，数据层人工评审必查
- ▶ R3 输入必校验：schema 校验一切外部输入——路径穿越、伪装扩展名、超长文本提示都是攻击面
- ▶ R4 密钥不入库：密钥走环境变量，git 历史扫描兜底——一旦入库，删文件没用，必须换密钥

纪律：凡 AI 写的日志/异常输出，用「这副代码会进生产吗」的眼镜看一遍；security 模块代码不接受 AI 主导生成。

等保基线的教学化映射：红线不是土规矩

对标等保二级

红线	对应等保 2.0 控制点（教学化）	SRS 条目	检查手段
R1 口令不明文	身份鉴别-口令复杂度与存储保护	SEC-01	脚本+认证模块人工评审
R2 SQL 参数化	安全计算环境-数据完整性/防注入	SEC-02	脚本+数据层人工评审
R3 输入必校验	安全区域边界-边界防护/入侵防范	SEC-02	契约扫描+schema 评审
R4 密钥不入库	安全管理制度-密码与密钥管理	SEC-01	脚本+git 历史扫描

红队模式：让 AI 扮演攻击者对本组接口穷举攻击向量（注入/越权/畸形输入），输出攻击清单，人工转化为红线扫描规则 —— 脚本扫模式，人审意图。

6.7

性能测试实操：一次完整的优化闭环

学完本节后，你应当能够：

- 目标：会立基线、会掐表、会用数据说服评审
- 案例：PE-01 交互分割响应从 3.4s 优化到 2.8s（预算 3000ms）

6.7 四纪律

性能测试的正确姿势：四条纪律 + 手工四步

环境 · 统计 · 基线 · 预算

- ▶ **环境纪律**：固定测试机（GPU 型号/驱动登记），本地数据只做参考 —— 本地 2.5s、测试机 3.6s 的教训
- ▶ **统计纪律**：交互类 50 次抽样取 P95，不报平均值 —— 平均值掩盖长尾抖动
- ▶ **基线纪律**：perf_history.csv 逐构建留档，性能趋势图进周报 —— 指标没人看等于没测
- ▶ **预算纪律**：每条性能指标标注需求条款出处（PE-01 ≤ 3s），无出处即打回

手工四步：定环境与数据集 → 跑基线留档 → 分段打点定位热点 → 优化后同环境复测对比。

数据驱动优化闭环

样例：3.4s → 2.8s 全记录

- ▶ **立基线**：首测 3.4s 超预算 —— 先记录，不猜原因
- ▶ **打点定位**：分段计时发现预处理占 1.9s（分块重叠区重复推理，即 D-04 缺陷）
- ▶ **优化**：重叠区结果复用 + 常驻 Worker 预热，同环境复测 2.8s 达标
- ▶ **防回归**：性能与精度同 PR 回归 —— 速度快了但 AT-01 降了，照样打回
- ▶ **AI 协同**：AI 估算性能「打八折」再采信（它对常数项系统性乐观）；优化方案 AI 出候选、人工按 profiling 数据裁决

常见问题三则：P95 抖动大 → 查 GC/缓存冷启动；本地快服务器慢 → 以测试机为准；指标没人看 → 挂进周报模板。

6.8

精度测试：冻结影像集与哈希登记制度

学完本节后，你应当能够：

- 目标：解决「输入漂移」，让精度结论可复现、可背书
- 案例：CR-001 阈值 0.30→0.24 的精度影响评估全链

先冻结输入，再谈指标：精度测试三要素

冻结 · 哈希 · 口径

- ▶ **冻结影像集**：固定 5 景影像版本锁定，任何人不得替换 —— 输入在变，指标就永远说不清
- ▶ **哈希登记**：每景影像 SHA-256 进登记册，运行前校验 —— 「数据被换过」这类争议用哈希一句话终结
- ▶ **口径写进 spec**：完整率/正确率按面积加权（不是图斑计数平均），判定边界规则写进 spec.md —— D-07 就是 AI 默认实现了全局口径
- ▶ **人工背书**：精度结论由人对采样结果签字，AI 只做计算与制图 —— 指标口径的解释权永远在人

输入漂移不解决，精度测试就是在测运气。

阈值变更必须数据驱动：CR-001 精度影响评估

0.30 → 0.24 的数据论证

- ▶ **问题：**T-18 联调实测发现 0.30 档 5 景中 2 景召回不足（漏检）—— 变更动因来自实测，不是「感觉更好」
- ▶ **实验：**冻结影像集上对 0.24/0.27/0.30 三档跑全量精度，面积加权口径计算
- ▶ **数据：**0.24 档完整率 76.8%→84.2%（达标），正确率 91.5%→88.6%（≥85% 判过线，仍达标）
- ▶ **决策：**CCB 有条件通过—— 四步闭环 + 精度复测归档后变更单方可关闭
- ▶ **归档：**复测数据进精度台账，RTM 行基线版本同步，旧值 0.30 全局清零

这条链路是「测试驱动变更」的完整范本：动因 → 实验 → 数据 → 决策 → 归档，缺一环就是拍脑袋。

6.9

降级与故障演练：把预案变成肌肉记忆

学完本节后，你应当能够：

- 目标：PE-09（GPU 不可用 30s 内降级 CPU、任务不丢）实测达标
- 没有演练过的降级方案，首次真实触发约有一半不能按预期工作

走查 → 实测 → 复盘：kill Worker 演练全记录

简化版混沌工程

- ▶ **板斧一 走查**：不开机，对着架构图走故障剧本，每步说出依据——走查发现「Worker 死后 in-progress 任务无回收逻辑」，任何单测都测不出的设计漏洞 (D-05)
- ▶ **板斧二 实测**：测试环境真杀 GPU Worker 进程掐表：心跳 2 周期 (10s) 发现失联 → 退避重排 (30/60/120s) → CPU 接管 → 任务零丢失；三次实测 18s/22s/25s，满足 PE-09 ≤30s
- ▶ **板斧三 复盘**：演练记录表五栏（剧本/预期/实际/偏差/改进项），改进项挂进 tasks.md——首演发现用户端无降级提示 (CHAOS-001)，修复后二演全绿

验收证据三样齐全才算过：演练记录表 + 实测时间线 + 缺陷修复闭环记录。

故障剧本库与 AI 协同：危险脚本必须逐行审

剧本库 · AI 穷举 · 审查纪律

- ▶ **剧本库**：每组至少抽演 3 个 —— GPU 不可用（必演）、模型依赖被卸载、磁盘写满、进程强杀后状态一致性、畸形输入（路径穿越/伪装扩展名）
- ▶ **按架构重编**：你的系统里不存在的组件，演它没有意义；剧本卡只写「故障现象」，预期行为从 spec/设计文档自己推导 —— 推导不出来说明文档没写清
- ▶ **AI 穷举故障模式**：每个组件问死了/慢了/返回错误数据，每种依赖间断连/超时/数据损坏，AI 穷举人工筛选 10~15 个
- ▶ **危险脚本纪律**：kill 进程/造毒消息脚本让 AI 写，但必须沙箱先跑、逐行人工审 —— AI 曾把 kill 过滤条件写宽，差点误杀宿主机进程

节奏：单元测试随 PR 跑，集成测试每日跑，故障演练每迭代至少一轮。

练习：为本组系统设计一个故障剧本

 选择本组架构中「故障时才执行」的一条路径（降级/重试/恢复之一），按三板斧设计并执行一次完整演练。可参考澜图 kill Worker 剧本的结构。

任务步骤

1. 写剧本卡：只写故障现象，预期行为从本组 spec.md 推导（写明条款编号）
2. 走查一遍：团队围坐对着架构图过剧本，记录发现的逻辑漏洞
3. 实测一遍：测试环境真注入故障，掐表记录时间线与任务终态
4. 填演练记录表：剧本/预期/实际/偏差/改进项，改进项挂 tasks.md

完成标准（可检查）

- ✓ 演练记录表五栏齐全
- ✓ 至少发现 1 个偏差并登记
- ✓ 无偏差须写明推导依据
- ✓ 切换类指标有实测秒数

6.10

调试方法论与重构技术债

学完本节后，你应当能够：

- 目标：先复现再修，假设按成本排序，修复必带回归用例
- AI 时代的 bug 更多是「理解偏了需求」与「组件间假设不一致」

6.10 先复现

调试第一原则：没有稳定复现，不许开始修

P-10 最小复现 · P-09 假设验证

- ▶ **最小复现**：能稳定触发问题的最小输入与最少步骤 —— P-10 提示词让 AI 协助缩小复现：提出 3 个最小化方案，各说明排除了哪些变量
- ▶ **成本对比**：样例统计，有最小复现的 bug 平均修复 1.8 小时，没有的平均 5.5 小时
- ▶ **假设-验证四步（P-09）**：列假设（每条写明「成立应观察到什么」）→ 按验证成本排序（先查日志后改代码）→ 逐条设计二值实验（证实/证伪，不许「好像是这样」）→ 修复与防复发（回答「为什么测试没拦住」）
- ▶ **教学案例**：精度测试偶发失败，AI 列 5 个假设，按成本排序后第 2 个实验定位到「并发写导致掩膜文件截断」—— 按直觉先怀疑算法至少浪费半天

靠猜的修复，十有八九引入新 bug。

与 AI 协作调试的三条纪律

三纪律 + 调试记录

- ▶ **日志喂全不喂截**：截段日志诱导 AI 瞎猜 —— 喂完整上下文，明确标注现象发生的时间点
- ▶ **AI 诊断是假设不是结论**：AI 说「问题是 X」时登记为假设之一，实验验证后再动手 —— 未经实验验证就改代码，评审一票否决
- ▶ **修复必带回归用例**：每个 bug 修复 PR 附一个「没有本修复就会失败」的测试 —— 它是修复正确性的永久证据，也是防复发哨兵
- ▶ **长调试留记录**：超过 2 小时的调试必须留记录（现象/假设清单/验证过程/根因/修复/防复发），格式与翻车记录兼容 —— 29 号文的 FR-02 就是由调试记录升级而来

调试记录是翻车记录的原料库。

技术债登记制度：债不可怕，不登记的债才可怕

登记五字段 · 三问偿还。样例如下：

编号	债务	产生原因	偿还时机与结果
TD-02	影像分块函数 260 行，圈复杂度超标	迭代一赶进度先求能跑	迭代二偿还：拆成 4 个函数（PR#61）
TD-05	模型加载测试用桩替身，与真实权重加载行为有差异	测试环境 GPU 未就绪	任务 4 末偿还：换 fakeredis+契约测试
TD-08	前端阈值组件与后端解析的单位约定靠口头	CR-001 前的历史遗留	CR-001 中一并根治（写进 OpenAPI）

登记五字段：位置/债务内容/产生原因/不还的代价/建议偿还时机（样例共登记 9 笔，偿还 5 笔，4 笔留任务 5 前清零）。三问决策法：债在下一代要动的路径上吗？利息在增长吗？影响验收指标吗？

AI 辅助重构的四条安全姿势

补网 · 小步 · 人控节奏 · 实证

- ▶ **先补网再重构**：目标代码覆盖率达标（核心 85%）才动手，不够先补测试——TD-02 先补 11 个用例再拆函数；测试不全的地方重构就是雷区跳舞
- ▶ **小步走，每步可验证**：大重构拆成一串小 PR，每个 PR 只做一种手法（提取函数/改名/搬文件），CI 绿一个再走下一个
- ▶ **AI 出方案，人控节奏**：AI 给步骤序列，人工审排序（先内层后外层）；每个手法单独会话——上下文一乱，AI 会把「改结构」和「改行为」混在一起
- ▶ **行为保持实证**：重构 PR 必附三件套——同一测试套件前后全绿、同一性能基线无退化、契约测试无变化；缺一评审不批

重构四个反模式：每个都有事故背景

识别与规避

反模式	表现	后果
顺手重构	修 bug 的 PR 里夹带重构	评审无法聚焦，修复与结构变化纠缠，回退困难
大爆炸重构	一个 PR 重写整个模块	测试网罩不住的边角全部暴露（PR#41 险些如此）
AI 全权重构	让 AI 一次性「优化这个模块」整体提交	行为变化无法归因，样例项目明令禁止
债不登记	口头说「这里以后要改」	迭代末无人记得，验收前暴雷

判断重构 PR 是否合格的一票标准：能不能用「行为保持三件套」证明外部行为零变化。

6.11

敏捷仪式：站会、周报与燃尽图

学完本节后，你应当能够：

- 目标：三个仪式做实 —— 站会 15 分钟硬上限，周报四段式，燃尽图真能预警
- 教材：32 号文两则真实纪要、27 号文两轮燃尽数据

站会三条做实规则，周报四段面向读者

三规则 · 四段式

- ▶ **对着看板说**：每人指着任务卡发言，状态当场更新 —— 凭记忆汇报的站会一周内流于形式
- ▶ **阻塞当场指派**：每个阻塞当场得到处理方案（谁帮/何时/升级教师） —— D6 站会「GPU 环境被占」当场改夜间跑精度，当天解决
- ▶ **超时转单聊**：技术讨论超 2 分钟记为会后单聊 —— 15 分钟是硬上限；合格纪要 11 行足矣
- ▶ **周报四段式**：进展（对照迭代目标+PR 号证据，禁「基本完成」） → 度量（PR 数/一次通过率/CI 拦截/覆盖率小表） → 问题与风险（必须人工写，诚实度是评分点） → 下周计划（从 tasks.md 拉任务带工时）

AI 可做速记与周报初稿，但「问题与风险」段必须人写 —— AI 倾向于把问题写得轻描淡写。

燃尽图：让它真能预警，而不是事后补画

取数 · 信号 · 动作

- ▶ **每日自动取数**：从看板真实状态脚本生成，杜绝手工美化——事后补画的曲线过分平滑，一眼假
- ▶ **平台期信号**：连续两天剩余量不降 = 任务粒度太大或估计失真——迭代一 D4 平台期（62h 两天未降），原因是交互分割估 8h 实花 22h，燃尽图提前暴露了估算偏差
- ▶ **偏离 30% 动作**：迭代中期剩余高于理想线 30% 以上，开风险会砍范围（教学场景不加班）
- ▶ **尽头必须归零**：迭代结束还有剩余任务 = 范围管理失败或状态不实，评审时必须解释

燃尽图的价值不在图，在「哪一天发现偏差的」。

仪式反模式清单：对照自查

五个反模式

仪式	反模式	症状	纠正
站会	背书式汇报	只报做了什么，不报阻塞	强制三问，阻塞必答
站会	技术讨论会	一个问题聊 20 分钟	2 分钟规则，转单聊
周报	流水账	罗列动作无结果无度量	四段式模板，无量词扣回
燃尽图	事后补画	曲线过分平滑	脚本自动取数，每日更新
燃尽图	永不归零	迭代末挂着一堆「快完成」	结束日强制清点，剩余进复盘

仪式做实的检验方法：随机抽一天，让任一成员复述当天站会的阻塞项与处理方案 —— 答不上来说明仪式是演给教师看的。

6.12

变更管理与 RTM v4 回填

学完本节后，你应当能够：

- 目标：背得出四步流程及顺序理由，会填变更单全字段
- 样例：31 号文 CR-001（文本提示阈值默认值 0.30→0.24）

CR-001 变更单逐字段：影响面与回退方案是重点

31 号文逐字段

字段	CR-001 实际内容	填写要点
编号与标题	CR-001 文本提示分割置信度阈值默认值调整	编号连续，标题说清改什么
提出人与日期	后端主攻，迭代二 D3（源于 T-18 联调实测）	任何人可提，但必须有记录
变更动因	冻结影像集初测 0.30 档 5 景中 2 景召回不足	数据驱动附证据，禁「感觉更好」
影响面清单	spec AC-FR-04.1-a、SRS FR-04.1、DLD 5.2、前后端代码、用例基线	逐项列出，是四步流程的导航图
数据论证	完整率 76.8%→84.2%、正确率 91.5%→88.6%（≥85% 判过线）	对比数据是批准的依据
回退方案	常量改回 0.30 即回退，无数据迁移	每个变更必须想好后路
批准记录	CCB 有条件通过：四步闭环+精度复测归档后方可关闭	组内 CCB，重大变更升级教师

为什么必须走流程：口头变更三后果 —— spec 与代码漂移（AI 按旧 spec 生成必然回退）、测试基线失效、验收争议（评委按 SRS 验收）。

四步流程逐步操作与闭环验证

spec → SRS → plan/tasks → 代码

- ▶ **第 1 步 改 spec**: 先改 spec.md 验收标准 (AC-FR-04.1-a 阈值与 Given/When/Then) , 评审通过 —— spec 是源头, 必须先于代码
- ▶ **第 2 步 同步 SRS**: FR-04.1 条款同步修改, 版本 V3.0→V3.1, 修订记录加一行
- ▶ **第 3 步 改 plan/tasks**: 新增 T-37 (0.25 人日依赖 T-18) 排进迭代, 燃尽基线 +0.25 注明变更来源, 风险册加 R-09
- ▶ **第 4 步 改代码**: PR#52 (DEFAULT_THRESHOLD 0.30→0.24, 注释引用 CR-001 与 SRS V3.1; 用例基线同步, 先红后绿)
- ▶ **闭环验证三项**: 全局搜旧值 0.30 清零 + 精度复测达标归档 + RTM 行基线版本同步 —— 三项打勾 CCB 签字才归档

反例: 某组口头把分页 20 改 50 只改后端 → 前端滚动异常、契约没覆盖、验收演示穿帮、三处不一致说不清哪个正确 → 配置管理项扣分。

RTM v4 九列结构：一条需求的生命周期

33 号文列结构

列	内容	填写时机	示例（FR-04.1）
需求编号	FR/NFR 编号	任务 2	FR-04.1
需求摘要	一句话	任务 2	文本提示分割与阈值管理
验收标准	AC 编号	任务 2	AC-FR-04.1-a/b
设计章节	HLD/DLD 章节号	任务 3	DLD 5.2 文本分割模块
代码位置	文件:类/函数	任务 4	text_prompt.py:DEFAULT_THRESHOLD
实现 PR	PR 编号	任务 4	PR#48~52, PR#52（CR-001）
测试用例	用例编号	任务 3/4	TC-F04-01~05, PT-03, AT-01
测试状态	通过/阻塞/未测	任务 4 持续	通过（夜间构建，AT 复测归档）
变更记录	CR 编号	随时	CR-001

回填四步：PR 合并即时登记（攒到迭代未必失真）→ 代码位置靠 docstring 的「Implements: FR-xx」+ check_refs 反向校验 → 测试状态从夜间构建取数 → 迭代评审前全量体检。三种诊断：正向（需求→实现）、反向（代码→需求抓孤儿代码）、变更波及（按 RTM 生成影响面骨架）。

6.13

代码文档同步与合格仓库解剖课

学完本节后，你应当能够：

- 目标：spec/SRS/OpenAPI/RTM/代码五件对齐，会防会查会修
- 标准：任务 4 验收时代码仓检查的对照样板

不一致的三个发生通道与各自防线

防 · 查 · 修

- ▶ **通道一 代码先行**：改了代码没改文档（最常见偷懒）—— 防线：变更四步流程强制 spec 先行
- ▶ **通道二 AI 回写**：AI 按旧 spec 快照生成覆盖新逻辑 —— 防线：context pack 每次现做，不用库存
- ▶ **通道三 并行漂移**：两人同时改关联工件各对一半 —— 防线：修改窗口站会声明，关联改动串行化
- ▶ **三个自动化检查**：contract_check.py（代码 ↔ OpenAPI，每次 CI）+ check_refs.py（编号双向闭合，每次提交）+ P-12 语义级扫描（AI 每周，查「退避 30/60/120 vs 指数退避」式漂移，每周扫出 2~4 条疑点约一半是真漂移）
- ▶ **D10 五件体检**：spec ↔ SRS 条款对应、契约全绿、check_refs 全绿、每条 AC 有用例且最近构建通过、变更旧值全局清零

修复改到源头：代码错了改代码，文档错了改文档 —— 禁止「两边都改得含糊一点凑合对上」。

6.13 仓库结构

sprint_repo 解剖：每部分为什么在那里

样例：34 号交付物逐目录

sprint_repo/

README.md#十分钟跑通承诺：架构一页图+三步启动+selfcheck

rules/constitution.md#项目宪章：人与AI共同遵守

docs/#spec快照、ADR、联调单归档

src/api/queue/worker/geo/security/#各守其位

tests/unit/contract/perf/accuracy/#三分离+marker

scripts/contract_check/security_redline

check_refs/selfcheck/chaos/#四条防线

.github/workflows/#CI五道闸配置

评审看点：queue 看状态机完整性与退避参数出处；worker 看降级路径与心跳超时（PE-09）；geo 看坐标参考系（常识替代高发区）；security 逐条对四红线、不接受 AI 主导。

README 验收：请别组同学照做，卡住超 2 分钟处就是要改的地方。

6.14

全流程复盘与缺陷案例集

学完本节后，你应当能够：

- 目标：按时间线复述一个真实 PR 的五环旅程
- 样例：30 号文 —— 交互分割功能（FR-03.1/03.2）从任务卡到合并

PR#47 时间线：五环各阶段耗时与关键事件

样例：30 号文实录

阶段	耗时	关键事件
任务认领与准备	0.5 天	领 T-19；制作 context pack；开工仪式
AI 初稿生成	1 轮会话	P-04 四段式，先出实现计划，人工审计划后出代码
自查与本地验证	0.5 天	跑单测/契约/selfcheck；发现 2 处问题自修
提交 PR 与评审	1 天	两轮评审意见，7 条修改，补 3 个测试
CI 与合并	0.5 天	首次红灯（增量 76.2%

context pack 五段实物：任务卡 / spec 条款（AC-FR-03.1/03.2 含单轮 ≤3s 预算） / 接口签名 / 约定清单（CGCS2000 存储、EPSG:4326 输出、GeoJSON 回传、1024² 裁剪） / 禁止事项（不动 GpuQueue 接口、不新增依赖）—— 约 40 行 25 分钟。

评审 7 条意见：正好覆盖五查的每一类

7 条意见全记录

- ▶ **生成阶段纠偏**：第一轮实现计划把阈值设计成 0~100 整数 —— 违反约定清单（常识替代需求发作），审计划阶段就拦下，没让它进入代码
- ▶ **事实类 1 条**：rasterio 调用要求附文档链接，查后确认存在
- ▶ **边界类 2 条**：超大提示框性能上限、掩膜为空的返回约定
- ▶ **意图类 1 条**：掩膜矢量化为何放 API 层而非 geo 层 —— 答复认可，但理由写进代码注释
- ▶ **数值类 1 条**：裁剪 1024^2 要求注明 PE-01 链路预算出处
- ▶ **测试类 2 条**：补并发提交用例、补提示坐标越界用例

五查清单有效的实证：真实评审意见的分布恰好落满五个类别。

复盘：AI 时代的工时结构变了

20/15/45/20 工时结构

- ▶ 工时分布：准备 20% / 生成 15% / 自查+评审+返修 45% / CI 与合并 20% —— AI 把「写代码」压到最短，重心转移到验证与评审
- ▶ 四环证据链：context pack → 会话记录 → 评审对话 → CI 记录，全留档，答辩逐环可追溯
- ▶ 登记习惯的复利：迭代二 CR-001 变更时，RTM 与影响面清单直接定位到 FR-04 行的关联 PR 与用例 —— 良好登记在变更时兑现价值
- ▶ 增量门禁实证：76.2%→89.7% 补 3 个异常分支约 1 小时 —— 门禁 80%「够得着但不放水」

评审与验证占 65% 工时不是低效，是 AI 时代质量生产的正常结构。

编码期缺陷案例十则：按发现环节 × 根因组织

AI 相关 6 则 / 流程相关 4 则

编号	缺陷	发现环节	根因类别	预防机制
D-01	rasterio 编造 API 运行期崩溃	运行期	编造 API	R-07 评审规则
D-02	阈值单位前后端不一致（0~100 vs 0~1）	联调	常识替代需求	联调单+契约补例
D-03	队满返回 500 而非 429	联调	spec 未读细	契约响应码断言
D-04	分块重叠区重复推理	性能测试	缺优化意识	性能分段打点
D-05	Task 死信状态无回收路径	走查演练	设计漏洞	故障走查制度
D-06	降级时用户无通知	故障演练	通知只判启动态	演练剧本覆盖
D-07	精度统计误用全局口径	评审	AI 默认实现	指标定义喂原文
D-08	docstring 需求编号错行	check_refs	编号错配	脚本反向校验
D-09	日志含明文口令字段	红线扫描	AI 教学式代码	security_redline.py
D-10	超时参数与任务规模脱节	评审	数值无出处	凡数字必问出处

用法：评审培训（给症状猜定位思路）→ PR 前自查（我犯没犯这十类）→ 本组新缺陷按同格式补入，案例库随项目生长。

D-02 / D-05 / D-09：三则缺陷的完整链路

症状 → 根因 → 预防

- ▶ **D-02 阈值单位**：前端传 35、后端按 0~1 解析直接报错 —— 正确的代码实现了错误的约定，单端测试都过，只有联调能抓；预防：反常识约定进 OpenAPI 与 context pack
- ▶ **D-05 Task 死信**：状态机只画了快乐路径，无外部失效驱动转换 —— 预防：状态机评审必画「异常驱动转换」，设计评审清单常驻项
- ▶ **D-09 日志明文口令**：AI 用公开语料里「调试友好」的整体打印 —— 预防：红线脚本 + 纪律「日志一律过脱敏层，敏感字段白名单制」

三则共性：缺陷都不在算法里，在约定、状态与习惯里 —— 这正是评审五查存在的理由。

第四关小结：研发冲刺的五句话

1 门禁即质量

五环单 PR 成本约 25 分钟，拦在 PR 阶段的缺陷以分钟计修复 —— 门禁是投资不是开销

2 护栏靠 CI 不靠自觉

覆盖率三层门禁、契约双向比对、红线脚本 —— 全进流水线，失败即停

3 只信实测

性能掐表、精度冻结影像集、降级真杀进程 —— 三样证据齐全才算过

4 变更走四步

spec 是源头、代码在最后；闭环验证三项打勾才归档

5 AI 资产是复利

模式库复用、翻车记录固化规则 —— 工时重心从写代码转向验证与评审（20/15/45/20）

立即行动：下一步：封版签字，移交任务 5 —— 质量门难关

代码能跑只是起点，证据齐全才算交付。

下一章：第五关 质量门禁 —— 三级测试、八大质量特性、组间对抗测试